Security in Software Applications
Homework 2

Rando, 1985084

A.A. 2024/2025

Abstract

In this report, I will describe my results with fuzzing ImageMagick 7.0.8-12 (as of 2025, the oldest available version from the official GitHub page) using AFL++. This fuzzing tool is a still-under-development upgrade of the open-source software AFL.

1 American Fuzzy Lop plus plus (AFL++)

AFL++ (American Fuzzy Lop plus plus) stands as a prominent and highly regarded tool within the domain of fuzz testing. For this project, AFL++ was selected over its predecessor, AFL, primarily due to its enhanced stability and robust integration with Docker environments. The initial setup involved pulling the official Docker image using the command docker pull aflplusplus/aflplusplus. Subsequently, an interactive container was launched via docker run -ti -v /location/of/your/target:/src aflplusplus/aflplusplus, which mounted the target project's source code into the container's /src directory. Within this interactive terminal session, the project was then compiled with AFL++ instrumentation by executing the make all command, preparing it for subsequent fuzzing operations.

1.1 Difficulties in Setting Up the Tool

The initial setup and configuration of Flawfinder within a Dockerized environment required dedicated effort. However, the process was significantly streamlined by referring to the official documentation. This phase involved constructing a Dockerfile based on the AFL++ image, downloading the ImageMagick directly inside the container, and subsequently building the ImageMagick binaries within the container using AFL++. The compilation process necessitated iterative refinement, as certain configuration options, such as the requirement for static linking of the software, initially led to build errors.

Early challenges were identified concerning the fuzzer's operational speed and resource management. An initial suboptimal build configuration was found to impede fuzzer performance. Concurrently, ImageMagick's generation of temporary files frequently led to memory saturation within the container. This issue was mitigated by implementing a scheduled job to periodically purge these temporary files.

Further investigation revealed that the fuzzer's inactivity and lack of reported crashes stemmed from an improperly configured ImageMagick execution command. Diagnostic testing of the command outside the fuzzing context confirmed a configuration file error. Resolving this required a thorough analysis of the configuration command's output and a detailed examination of the ImageMagick source code.

A critical operational challenge emerged regarding container management and fuzzing session resumption. It was discovered that initiating the container directly with the fuzzing command prevented subsequent resumption, as the resume command differs and generates an error when existing fuzzing artifacts are detected, leading to container termination. This issue regrettably resulted in the loss of several hours of fuzzing data due to an unexpected crash. To circumvent this, subsequent fuzzing sessions were initiated by launching the container with a persistent command, such as `sleep infinity`, allowing the fuzzer to be manually executed and managed from within the running container.

Finally, the potential for leveraging multiprocessing to enhance fuzzing throughput was recognized belatedly.

After approximately four hours of operation, the host system experienced an unexpected and complete crash, despite monitoring indicating no excessive CPU utilization or other demanding background processes. Consequently, the fuzzing campaign was halted, deviating from the recommended run-time of at least 24 hours. Notably, unusual files were subsequently discovered within an unexpected directory inside the container, the implications of which warrant further investigation. Fortunately, all accumulated fuzzing data remained intact, and the container's status could be successfully re-evaluated.

1.2 Final Setup

In the finalized fuzzing setup, the target application was compiled with the necessary fuzzer instrumentation. The fuzzing campaign was initiated using an initial corpus of 10 input files, with the workload parallelized across 12 CPU cores to maximize efficiency. Furthermore, a scheduled cron job was configured to regularly clear temporary files generated during the fuzzing process, preventing disk space exhaustion.

The specific command utilized for fuzzing was `magick <input> <output>`, which serves to convert an input file using the ImageMagick utility. In this command, the `<input>` parameter was designated as `@@`, a placeholder that directs the fuzzer to supply its generated test cases. The `<output>` was redirected to `/dev/null`, a special device file in Unix-like operating systems that discards all data written to it. This redirection was crucial for preventing memory or disk space saturation by ensuring that the extensive output generated during fuzzing was immediately discarded.

Comprehensive details regarding the project's structure, implementation, and methodology are available on the dedicated GitHub page.

2 Results

Within approximately two hours of execution, each parallel fuzzing process successfully identified crashes and multiple instances of program hangs. The Table

core	0	1	2	3	4	5
mutations	2027620	1863739	1883427	1790680	1841432	1892278
saved crashes	3	1	1	12	1	2
saved hangs	99	86	92	67	73	77

	6	7	8	9	10	11
	1855224	1794054	1812614	1764810	1849085	1897920
	1	1	1	3	1	1
	67	94	77	79	77	79

Table 1: Fuzzing campaign results. Each core run for about 4 hours and started 11 files from the samples file used in another public project. (There is also a dedicated page, but the files appear to be corrupted to my Windows Defender.). Initially I've gathered a much bigger corpus, but after improving it with AFL-minimize it reduces to few files. More statistics can be found in the dedicated GitHub page for this project.

1 provide a detailed examination of every setting and parameter utilized in this fuzzing campaign (available in the `fuzzer_stats` created by AFL++).

The identified crashes were compared against the Common Vulnerabilities and Exposures (CVEs) listed on cve.mitre.org using a custom script, `checkCVE.sh`, to automate the detection of known vulnerabilities. Regrettably, these crashes do not appear to correspond to any pre-existing CVEs. It is hypothesized that a longer fuzzing duration, unhindered by the unexpected system crash, might have increased the likelihood of discovering novel vulnerabilities.

Figure 1: The execution of the command ls inside the container. Every file without an exetension was not expected and has surely been genetaed due tu bugs in ImageMagick.